Step 3 — The Pipeline — Solution

> Back to the step · Hint 1 · Hint 2

Story: `run_query` ties it together — index the repo, retrieve, build context, answer, attach cited files — plus `main.py` and the integration test. **If not:** the tested pieces never meet, and there's no single call that answers a codebase question.

Sample repo (`sample_repo/`)

```
# sample_repo/auth.py
from session import create_session


def login(user, password):
    if check_password(user, password):
        return create_session(user)
    return None


def check_password(user, password):
    return password == lookup_hash(user)
```

```
# sample_repo/session.py
SESSIONS = {}


def create_session(user):
    SESSIONS[user] = {"active": True}
    return SESSIONS[user]


def end_session(user):
    SESSIONS.pop(user, None)
```

```
# sample_repo/db.py
def connect():
```

```
    # open a pooled database connection
    return {"pool": "ready"}
```

(The `.py` samples are written as text blocks here; save them as real `.py` files in `sample_repo/`.)

The pipeline

```
# src/pipeline.py
# Index the repo, retrieve snippets for the question, build the grounded
# context, answer, and return the answer plus the files it came from.
from src.answer import answer_question
from src.context import build_context, cited_files
from src.rag.code_store import build_code_store
from src.rag.retriever import retrieve_snippets
from src.utils.logger import get_logger


def run_query(model, repo_dir: str, question: str):
    logger = get_logger(__name__)
    collection = build_code_store(repo_dir)
    snippets = retrieve_snippets(collection, question)
    context = build_context(snippets)
    answer = answer_question(model, question, context)
    files = cited_files(snippets)
    logger.info(f"answered from {len(files)} file(s)")
    return answer, files
```

main.py

```
# main.py
import sys

from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_query


def main():
    if len(sys.argv) < 2:
        print('usage: python main.py "your question"')
        return
    question = sys.argv[1]
```

```
    config = load_config()
    model = get_model(config)
    answer, files = run_query(model, config.repo_dir, question)

    print(f"\nAnswer:\n  {answer}")
    print("\nCited files:")
    for file in files:
        print(f"  - {file}")


if __name__ == "__main__":
    main()
```

The integration test (real model + RAG)

```
# tests/integration/test_live_query.py
# Real model + RAG. Run from the repo root:
#   python -m tests.integration.test_live_query
from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_query


def test_answers_from_the_code() -> None:
    config = load_config()
    model = get_model(config)
    answer, files = run_query(model, config.repo_dir, "How is login handled?")
    ok = "login" in answer.lower() and "auth.py" in files
    status = "OK" if ok else "WRONG"
    print(f"case 1 (login answered, auth.py cited) -> {status}")


def test_offtopic_says_unknown() -> None:
    config = load_config()
    model = get_model(config)
    answer, files = run_query(
        model, config.repo_dir, "How do I train a neural network here?")
    lowered = answer.lower()
    ok = "don't know" in lowered or "not" in lowered
    print(f"case 2 (off-topic -> unknown)         -> {'OK' if ok else 'WRONG'}")


if __name__ == "__main__":
```

```
    test_answers_from_the_code()
    test_offtopic_says_unknown()
```

Expected run:

```
$ python main.py "How is login handled?"

Answer:
  Login is handled in auth.py by login(user, password), which checks the
  password with check_password() and creates a session via create_session()
  in session.py.

Cited files:
  - auth.py
  - session.py
```

That's the whole project: the codebase indexed, the snippets most relevant to the question retrieved, and the model answering only from them — with the files it drew on cited, so the answer is both correct for this repo and traceable back to the source.